

on run

```
-- =====
-- 1. CHECK KEYBOARD (English ABC/US vs Vietnamese)
-- =====

try

    set currentInputSource to do shell script "defaults read
com.apple.HIToolbox AppleSelectedInputSources | grep
'\\"InputSourceID\\"' | head -n 1 | awk -F'\\"' '{print $4}'"

    if currentInputSource is "" then

        set currentInputSource to do shell script
"defaults read com.apple.HIToolbox AppleSelectedInputSources |
grep 'KeyboardLayout Name' | head -n 1 | awk -F=' ' '{print $2}'
| tr -d ' ;\\""

    end if

on error

    set currentInputSource to "Khong xac dinh"

end try

set validSources to {"com.apple.keylayout.ABC", "com.apple.keylayout.US",
"ABC", "US"}

set vietnameseSources to {"Telex", "telex", "VNI", "vni", "Unikey",
"Vietnamese", "VIQR"}

set isEnglish to false
repeat with validItem in validSources
    if currentInputSource contains validItem then
        set isEnglish to true
        exit repeat
    end if
end repeat

set isVietnamese to false
repeat with viItem in vietnameseSources
    if currentInputSource contains viItem then
        set isVietnamese to true
        exit repeat
    end if
end repeat

if isVietnamese or (not isEnglish) then
    display dialog "LOI BO GO: He thong phat hien [" &
currentInputSource & "]" & return & return & "Vui long chuyen sang
```

```

        ban phim 'ABC' hoac 'U.S.' (Tat Telex/VNI) truoc khi chay!" with
        title "Swift VM TurboTyper" buttons {"Da hieu"} default button "Da
        hieu" with icon stop

        return

    end if

-- =====
-- 2. CONFIRM
-- =====

    display dialog "Che do: Danh lua may ao bang Ma Phim Vat Ly." & return &
    return & "Ban co 3 giay de click vao VM." & return & return & "DUNG: giu
    phim ESC (hoac di chuot len goc TREN-TRAI man hinh)." with title "Xac nhan
    go tu dong" buttons {"Huy", "Bat dau"} default button "Bat dau" cancel
    button "Huy"

    set swiftCode to "import Foundation
import CoreGraphics
import Cocoa
// Emergency stop: mouse moved into top-left corner (x<50, y<50)
func isEmergencyStop() -> Bool {
    let loc = NSEvent.mouseLocation
    let screen = NSScreen.main?.frame ?? .zero
    return loc.x < 50 && (screen.height - loc.y) < 50
}
guard let clipboard = NSPasteboard.general.string(forType: .string) else {
exit(1) }
usleep(3_000_000) // 3s to click into VM
let source = CGEventSource(stateID: .hidSystemState)
let shiftKeyCode: CGKeyCode = 56
let keyMap: [Character: (CGKeyCode, Bool)] = [
    "\a": (0, false), \A": (0, true), \b": (11, false), \B": (11,
true),
    \c": (8, false), \C": (8, true), \d": (2, false), \D": (2,
true),
    \e": (14, false), \E": (14, true), \f": (3, false), \F": (3,
true),
    \g": (5, false), \G": (5, true), \h": (4, false), \H": (4,
true),
    \i": (34, false), \I": (34, true), \j": (38, false), \J": (38,
true),
    \k": (40, false), \K": (40, true), \l": (37, false), \L": (37,
true),

```

```

        \ "m\ ": (46, false), \ "M\ ": (46, true), \ "n\ ": (45, false), \ "N\ ": (45,
true),
        \ "o\ ": (31, false), \ "O\ ": (31, true), \ "p\ ": (35, false), \ "P\ ": (35,
true),
        \ "q\ ": (12, false), \ "Q\ ": (12, true), \ "r\ ": (15, false), \ "R\ ": (15,
true),
        \ "s\ ": (1, false), \ "S\ ": (1, true), \ "t\ ": (17, false), \ "T\ ": (17,
true),
        \ "u\ ": (32, false), \ "U\ ": (32, true), \ "v\ ": (9, false), \ "V\ ": (9,
true),
        \ "w\ ": (13, false), \ "W\ ": (13, true), \ "x\ ": (7, false), \ "X\ ": (7,
true),
        \ "y\ ": (16, false), \ "Y\ ": (16, true), \ "z\ ": (6, false), \ "Z\ ": (6,
true),
        \ "1\ ": (18, false), \ "! \ ": (18, true), \ "2\ ": (19, false), \ "@ \ ": (19,
true),
        \ "3\ ": (20, false), \ "# \ ": (20, true), \ "4\ ": (21, false), \ "$ \ ": (21,
true),
        \ "5\ ": (23, false), \ "% \ ": (23, true), \ "6\ ": (22, false), \ "^ \ ": (22,
true),
        \ "7\ ": (26, false), \ "& \ ": (26, true), \ "8\ ": (28, false), \ "* \ ": (28,
true),
        \ "9\ ": (25, false), \ "( \ ": (25, true), \ "0\ ": (29, false), \ ") \ ": (29,
true),
        \ "- \ ": (27, false), \ "_ \ ": (27, true), \ "= \ ": (24, false), \ "+ \ ": (24,
true),
        \ "[ \ ": (33, false), \ "{ \ ": (33, true), \ "]" \ ": (30, false), \ "} \ ": (30,
true),
        \ "\\ \ \ \ ": (42, false), \ "| \ ": (42, true), \ "; \ ": (41, false), \ ": \ ":
(41, true),
        \ "' \ ": (39, false), \ "\" \ \ \ ": (39, true), \ ", \ ": (43, false), \ "< \ ":
(43, true),
        \ ". \ ": (47, false), \ "> \ ": (47, true), \ "/" \ ": (44, false), \ "? \ ": (44,
true),
        \ "` \ ": (50, false), \ "~ \ ": (50, true), \ " \ ": (49, false),
        \ "\ \ n \ ": (36, false), \ "\ \ r \ ": (36, false), \ "\ \ t \ ": (48, false)
]
var n = 0
for char in clipboard {
    n += 1
    // STOP HOTKEY: hold Esc (keycode 53) to abort immediately
    if CGEventSource.keyState(.combinedSessionState, key: 53) { exit(0) }
}

```

```

// Backup stop: mouse into top-left corner
if n % 15 == 0 && isEmergencyStop() { exit(0) }
var lookupChar = char
if char == "\\u{201C}" || char == "\\u{201D}" { lookupChar =
\\"\\\" }
if char == "\\u{2018}" || char == "\\u{2019}" { lookupChar = "'" }
}
if char == "\\u{2013}" || char == "\\u{2014}" { lookupChar = "-"}
}

guard let mapping = keyMap[lookupChar] else { continue }
let code = mapping.0
let needsShift = mapping.1
if needsShift {
    let shiftDown = CGEvent(keyboardEventSource: source,
virtualKey: shiftKeyCode, keyDown: true)
    shiftDown?.flags = .maskShift
    shiftDown?.post(tap: .cghidEventTap)
    usleep(12_000) // give the VM time to register the modifier
}
let keyDown = CGEvent(keyboardEventSource: source, virtualKey: code,
keyDown: true)
if needsShift { keyDown?.flags = .maskShift }
keyDown?.post(tap: .cghidEventTap)
usleep(4_000)
let keyUp = CGEvent(keyboardEventSource: source, virtualKey: code,
keyDown: false)
if needsShift { keyUp?.flags = .maskShift }
keyUp?.post(tap: .cghidEventTap)
if needsShift {
    usleep(6_000) // hold shift a bit before releasing
    let shiftUp = CGEvent(keyboardEventSource: source,
virtualKey: shiftKeyCode, keyDown: false)
    shiftUp?.post(tap: .cghidEventTap)
    usleep(4_000)
}
usleep(code == 36 ? 25_000 : 3_000)
}

```

```

do shell script "printf %s " & quoted form of swiftCode & " | swift -"
end run

```

```

use framework "AppKit"
use scripting additions
-- ===== CONFIG =====
property chunkSize : 12 -- chars per burst (smaller = safer)
property chunkDelay : 0.03 -- pause between bursts (s)
property lineDelay : 0.08 -- pause after Enter (s)
property startDelay : 3 -- countdown before typing (s)
-- Abort hotkey: hold Control + Option (^⌘)
on run
    -- 1) Keyboard layout guard
    set currentInputSource to getInputSource()
    set okLayouts to {"com.apple.keylayout.ABC", "com.apple.keylayout.US", "ABC",
"US"}
    if okLayouts does not contain currentInputSource then
        set userChoice to display dialog ¬
            "CẢNH BÁO BỘ GỖ: Hệ thống nhận diện [" & currentInputSource & "]"
        & return & return & ¬
            ¬
            "Nếu đang bật Tiếng Việt, text sẽ bị loạn dấu. Hãy chuyển
về US/ABC!" with title "Kiểm tra bộ gõ" buttons {"Hủy", "Vấn gõ"} default button
"Hủy" with icon caution
        if button returned of userChoice is "Hủy" then return
    else
        display dialog ¬
            "Gõ siêu tốc bằng AppleScript cho Viettel." & return & return & ¬
            "Giữ ^⌘ (Control+Option) bất cứ lúc nào để DỪNG." & return & ¬
            "Nhấn 'Bắt đầu', bạn có " & startDelay & ¬
            "s để click vào máy ảo." with title "Xác nhận gõ siêu tốc" buttons
{"Hủy", "Bắt đầu"} default button "Bắt đầu" cancel button "Hủy"
    end if

    -- 2) Type
    set clipboardText to (the clipboard as text)
    set textLines to paragraphs of clipboardText
    set lineCount to count of textLines

    delay startDelay

    repeat with i from 1 to lineCount
        if isAbortHeld() then
            beep

```

```

        display notification "Đã dừng tại dòng " & i with title "Auto
Type"

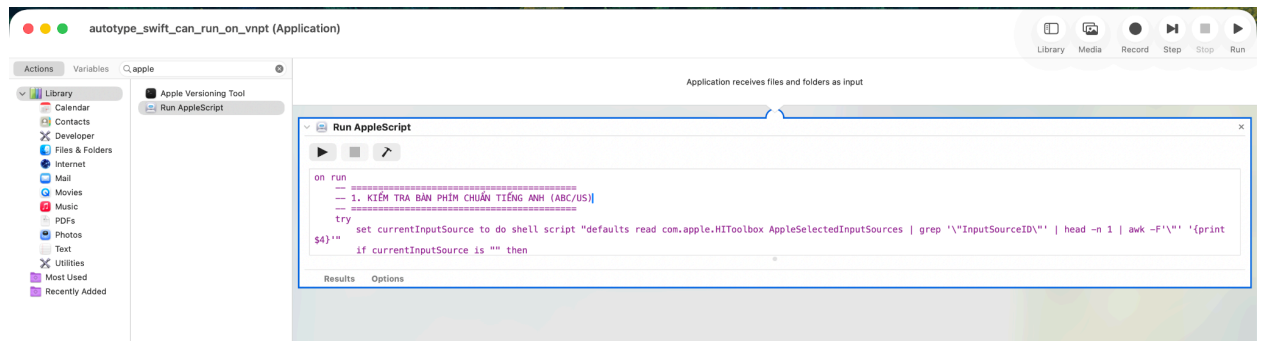
        return
    end if
    set thisLine to (item i of textLines) as string
    if thisLine is not "" then typeChunked(thisLine)
    tell application "System Events" to key code 36 -- Enter
    delay lineDelay
end repeat
display notification "Hoàn tất " & lineCount & " dòng" with title "Auto Type"
end run
-- Type a string in small bursts, checking the abort hotkey between bursts.
on typeChunked(theText)
    set theLen to count of theText
    set pos to 1
    repeat while pos ≤ theLen
        if isAbortHeld() then
            beep
            display notification "Đã dừng giữa dòng" with title "Auto Type"
            error number -128 -- user cancelled
        end if
        set endPos to pos + chunkSize - 1
        if endPos > theLen then set endPos to theLen
        set chunk to text pos thru endPos of theText
        tell application "System Events" to keystroke chunk
        set pos to endPos + 1
        delay chunkDelay
    end repeat
end typeChunked
-- True if Control + Option are currently held (polled globally via NSEvent).
on isAbortHeld()
    set f to (current application's NSEvent's modifierFlags()) as integer
    set ctrlHeld to ((f div 262144) mod 2) = 1 -- NSEventModifierFlagControl
    set optHeld to ((f div 524288) mod 2) = 1 -- NSEventModifierFlagOption
    return (ctrlHeld and optHeld)
end isAbortHeld
on getInputSource()
    try
        set src to do shell script "defaults read com.apple.HIToolbox
AppleSelectedInputSources | grep '\"InputSourceID\"' | head -n 1 | awk -F'\"' '{print
$4}'"
        if src is "" then

```

```
        set src to do shell script "defaults read com.apple.HIToolbox
AppleSelectedInputSources | grep 'KeyboardLayout Name' | head -n 1 | awk -F=' ' '{print
$2}' | tr -d ' ;\\""
        end if
    on error
        set src to "Không xác định"
    end try
    return src
end getInputSource
```

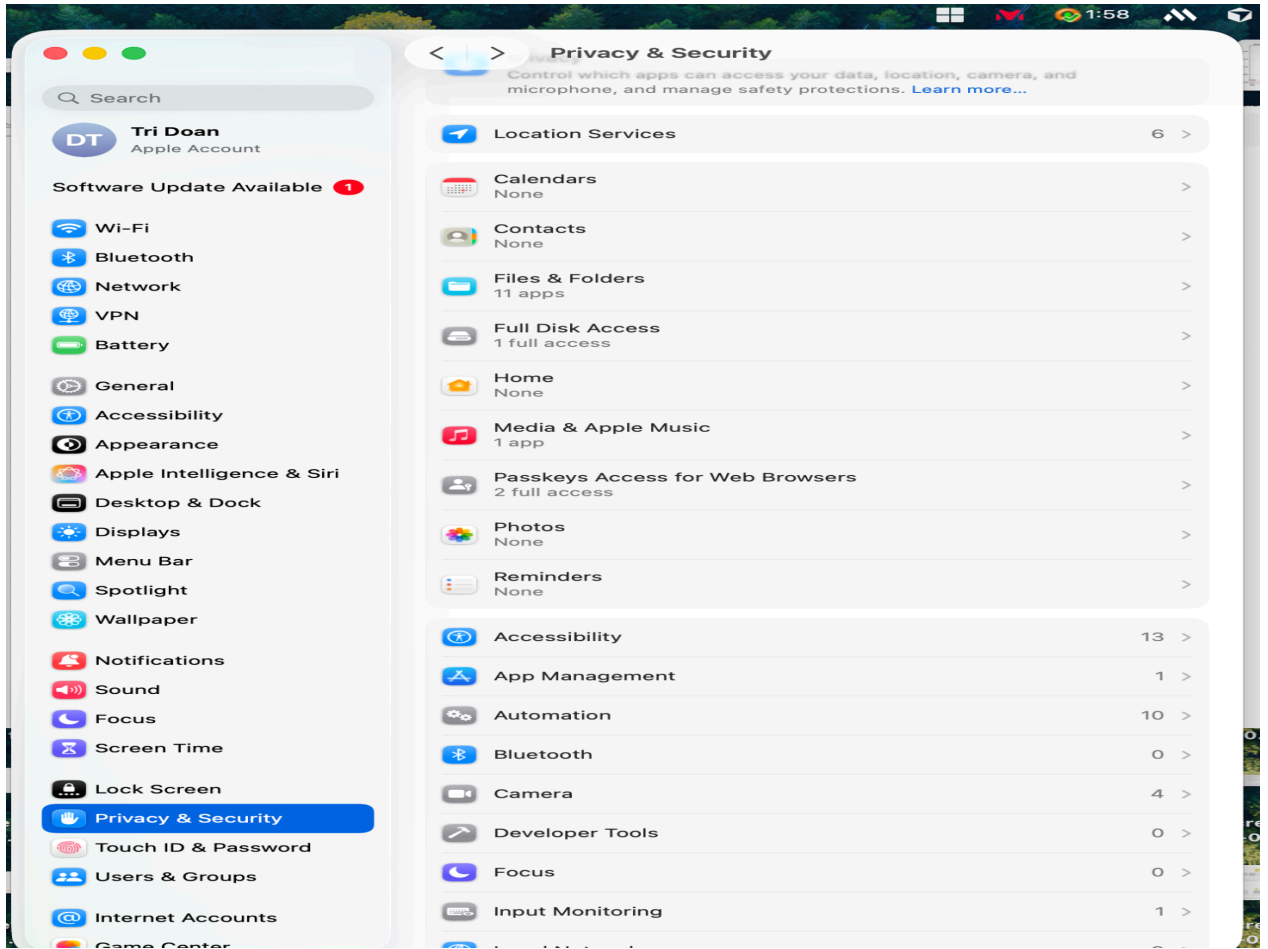
Bước 1:

- Mở automator lên, dùng cái apple script để thả code vào và lưu lại dưới dạng format application



Bước 2:

- Mở setting -> privacy and sec -> accessibility, cấp quyền cho mấy cái application vừa tạo
- Sau đó cấp quyền cho các app: app trình duyệt, app autotype từ code, automator, shortcut





Accessibility

Allow the applications below to control your computer.



Arc



Automator



autotype_by_cliclick_v2



autotype_by_keystroke_v2



autotype_swift_can_run_on_vnpt



cliclick_autotype



cliclick



Cursor



Google Chrome



Script Editor



Shortcuts



Slack



Terminal

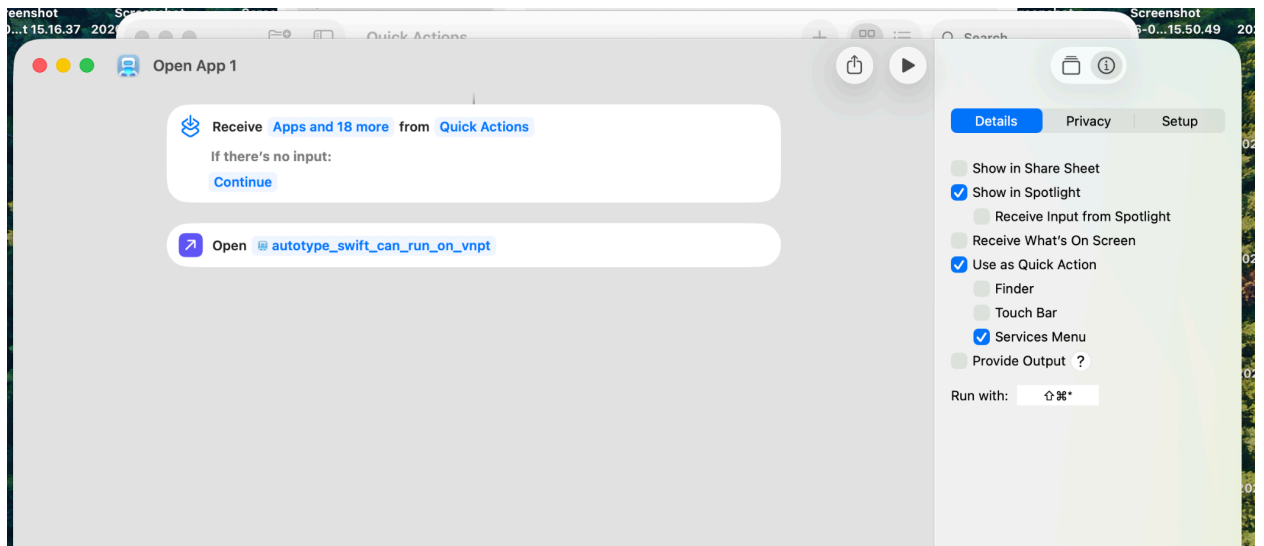


Windows App



Bước 3

- Vào shortcut thêm shortcut để open cái app autotype đó lên



Bước 4

- Paste thử vào gg doc hay đâu đó để xem còn thiếu quyền gì nữa không trước